

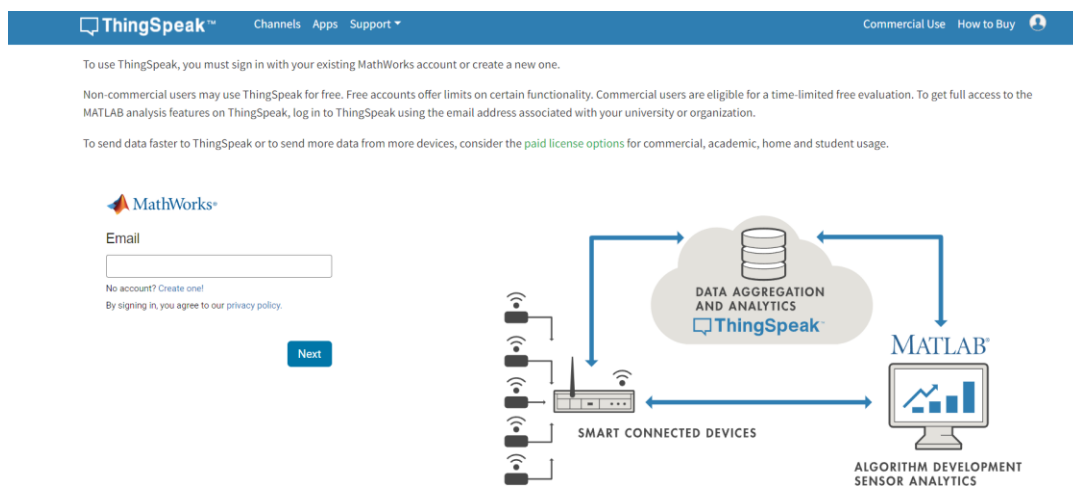
PROCEDURE

Step 1: Installation of Required Tools

1. Install Python (version 3.8 or above) and add it to the system PATH.
2. Using the command prompt, install the required Python libraries by typing the following commands:
`pip install numpy pandas tensorflow scikit-learn matplotlib`
3. Install STM32CubeIDE from STMicroelectronics for firmware development and AI model integration.



4. Ensure that the X-CUBE-AI package is available within CubeIDE for model conversion.
5. Install STM32CubeProgrammer for flashing firmware into the STM32 microcontroller.
6. Install Arduino IDE and configure it to support ESP32 boards by adding the following URL in Preferences:
`https://dl.espressif.com/dl/package_esp32_index.json`
7. Create a ThingSpeak account and set up a new channel.



- Enable Field 1 and name it “Engine Fault”.
- Copy the Write API Key for later use in the ESP32 program.

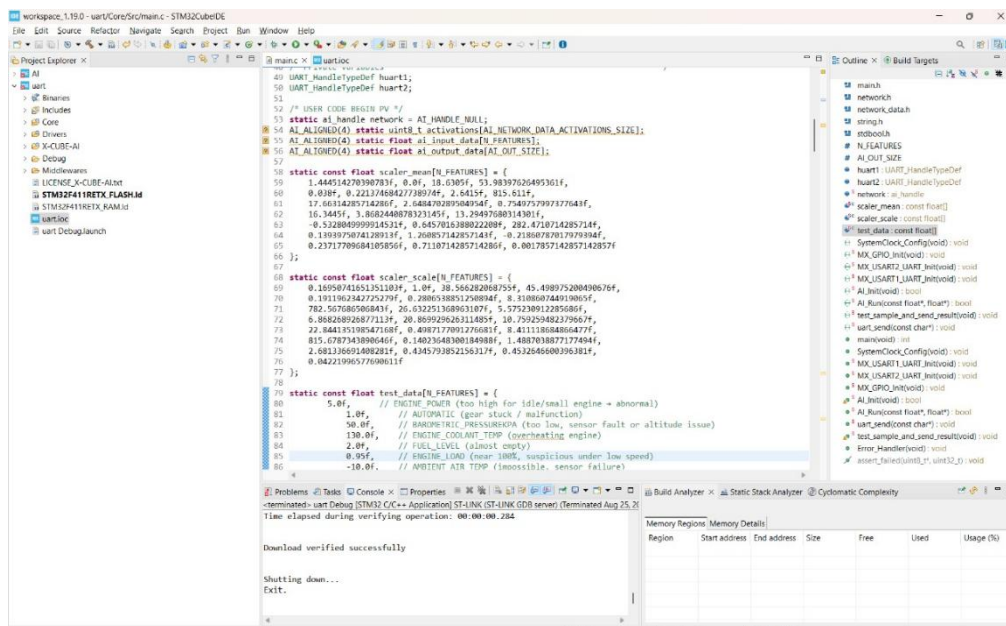
Step 2: Model Training and Preparation

1. Run the Python file named `train_stm32_mil.py` to train the machine learning model using the dataset `engine_balanced_dataset.csv`.
2. The model is trained to predict the Malfunction Indicator Light (MIL) value, where 0 represents a normal condition, and 1 represents an engine fault.
3. After successful training, the script automatically saves the output files in a folder named “artifacts”, which includes:
 - `trained_model.tflite` – the converted TensorFlow Lite model
 - `feature_meta.json` – information about feature scaling and mean values
 - `scaler_stats.npz` – normalization data

```
train_ml.py 9
F: > jot project > train_ml.py > ...
1 import numpy as np
2
3 import pandas as pd
4 from pathlib import Path
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import classification_report, confusion_matrix
8 from sklearn.preprocessing import StandardScaler
9
10 import tensorflow as tf
11 from tensorflow import keras
12 from tensorflow.keras import layers
13
14 CSV_PATH = "engine_balanced_dataset.csv"
15 df = pd.read_csv(CSV_PATH)
16
17
18 TARGET = "MIL"
19
20
21 feature_cols = [c for c in df.columns if c != TARGET]
22
23 X = df[feature_cols].astype(np.float32).values
24 y = df[TARGET].astype(np.int32).values
25
26
27 X_train, X_tmp, y_train, y_tmp = train_test_split(
28     X, y, test_size=0.30, random_state=42, stratify=y
29 )
30 X_val, X_test, y_val, y_test = train_test_split(
31     X_tmp, y_tmp, test_size=0.50, random_state=42, stratify=y_tmp
32 )
33 print(f"Shapes -> train:{X_train.shape}, val:{X_val.shape}, test:{X_test.shape}")
34
```

Step 3: Model Conversion and STM32 Integration

1. Open STM32CubeIDE and create a new project for the selected STM32 board (for example, Nucleo-F446RE).
2. In the Software Packs tab, open X-CUBE-AI and select Add AI Model. Import the trained_model.tflite file.
3. The Cube.AI tool automatically generates the required AI header files such as network.h, network_data.h, and network_data_params.h.
4. Configure the UART peripheral for communication as follows:
 - USART2 TX connected to ESP32 RX
 - USART2 RX connected to ESP32 TX
5. Configure one GPIO pin (for example, PA5) for LED indication.
6. Generate initialization code using STM32CubeIDE.



Step 4: Implementation of STM32 Code

1. Replace the generated main.c file with the custom code that includes the following operations:
 - Model initialization and memory allocation
 - Data normalization
 - AI model inference
 - UART data transmission to ESP32

- LED status indication based on the fault result
- 2. Build the project and flash the code to the STM32 microcontroller using ST-LINK.
- 3. Open the serial monitor to verify the output.
 - “Fault: 0” indicates a normal condition.
 - “Fault: 1” indicates an engine fault detected.

Step 5: ESP32 Configuration and Data Transmission

1. Open Arduino IDE and select ESP32 Dev Module from the board options.
2. Load the ESP32 program that performs the following tasks:
 - Reads fault status data from the STM32 via UART.
 - Converts the received message into a numerical format.
 - Sends this numeric value to the ThingSpeak cloud platform through Wi-Fi.
3. Before uploading, update the following parameters in the program:
Replace *your_wifi_name* with your Wi-Fi network name.
Replace *your_wifi_password* with your Wi-Fi password.
Replace *YOUR_API_KEY* with the Write API key from your ThingSpeak channel.
4. After entering the correct details, compile and upload the program to the ESP32 board using the Arduino IDE.
5. Once the upload is complete, the ESP32 automatically connects to the Wi-Fi network and transmits data to ThingSpeak at regular intervals.



The screenshot displays the Arduino IDE interface. The main editor window shows a C++ sketch for an ESP32 Dev Module. The code includes ThingSpeak API settings, variables for data processing, and logic for connecting to Wi-Fi and sending data to the cloud. The Serial Monitor at the bottom shows the output of the program, including the message "UART Receiver Started...", the IP address "10.109.110.17", and the received data "Fault: 1".

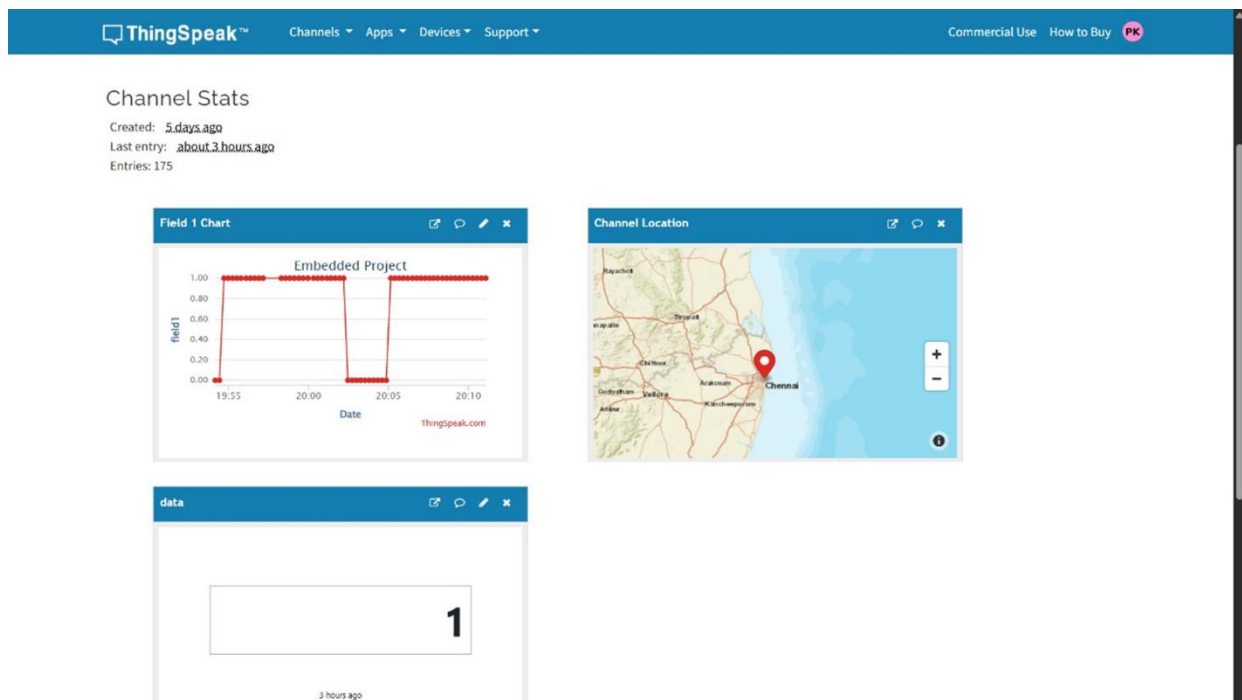
```
sketch_aug25a | Arduino IDE 2.3.6
File Edit Search Tools Help

ESP32 Dev Module

sketch_aug25a.ino
13 // ThingSpeak settings
14 const char* thingspeakApiKey = "CU00CRD9K47854QV";
15 const char* thingspeakServer = "http://api.thingspeak.com/update";
16
17 // Variables for data processing
18 String receivedData = "";
19 String processedValue = ""; // numeric data for ThingSpeak
20 String fullMessage = ""; // "Fault: 0" full text
21 unsigned long lastSendTime = 0;
22 const unsigned long sendInterval = 15000; // 15 sec (thingspeak limit)
23
24 // Wifi connection status
25 bool wifiConnected = false;
26
27 void setup() {
28   Serial.begin(115200);
29   Serial2.begin(UART_BAUD, SERIAL_BRI, RXD2, TXD2);
30
31   // Connect to WiFi
32   connectToWiFi();
33   Serial.println("UART Receiver Started...");
34 }
35
36
37 void loop() {
38   // Check wifi connection status
39   if (WiFi.status() != WL_CONNECTED) {
40     wifiConnected = false;
41     Serial.println("WiFi disconnected. Attempting to reconnect...");
42     connectToWiFi();
43   } else if (!wifiConnected) {
44     // ...
45   }
46 }
47
48 // Serial Monitor X Output
49 Message (Enter to send message to 'ESP32 Dev Module' on 'COM8') Both NL & CR 115200 baud
50
51 -----
52 RTD: Hello!
53 -----
54 WiFi connected successfully!
55 IP Address: 10.109.110.17
56 Signal Strength (RSSI): -33 dBm
57 UART Receiver Started...
58 Full message received: Fault: 1
59 Numeric value to send: 1
```

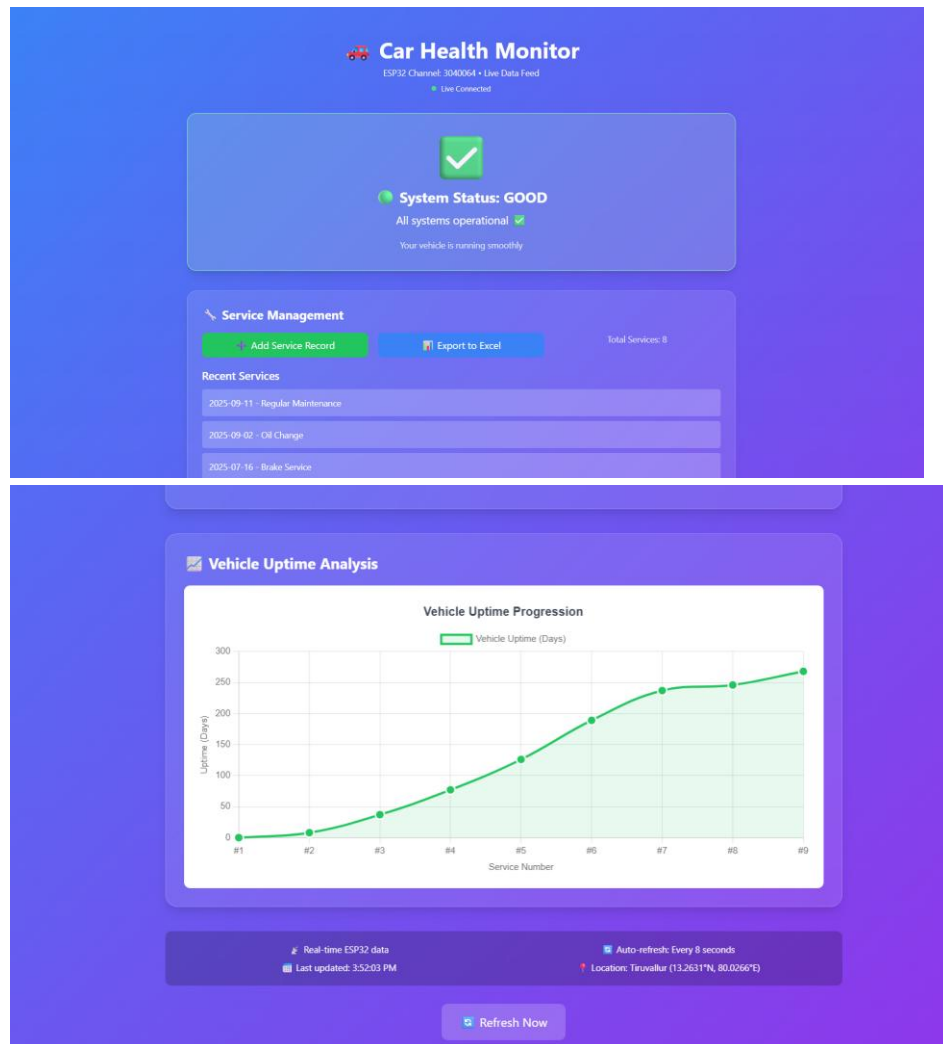
Step 6: Cloud Visualization and Dashboard

1. Log in to the ThingSpeak account and open the created channel.
2. Observe the real-time graph for the field “Engine Fault”.
 - Value 0 indicates normal condition.
 - Value 1 indicates fault detected.
3. The data can also be visualized through a customized user dashboard that includes:
 - An alert system to notify fault detection.
 - Google Maps integration to display the location of the faulty vehicle.



Step 7: System Testing and Validation

1. Simulate different engine input conditions and observe the system response.
2. Verify that:
 - The STM32 correctly performs real-time inference and updates the LED indicator.
 - UART communication between STM32 and ESP32 is stable.
 - The ESP32 uploads correct data to the ThingSpeak cloud.
 - The dashboard accurately displays real-time fault status.



Inference

The trained machine learning model successfully detects engine faults in real time. The STM32 microcontroller executes the AI model, processes the sensor input data, and transmits the output to the ESP32 via UART. The ESP32 module sends this data to the ThingSpeak cloud, where it is visualized on a user dashboard that integrates an alert system and Google Maps for location tracking.